

# Failure (bad key + cost comparison) — Solution

---

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume `client = OpenAI()` with `.env` already loaded.

## Basic Version

---

### Approach 1 — the direct way

```
# auth_and_cost_practice.py
import openai
from openai import OpenAI

client = OpenAI()

# part 1: bad key
bad_client = OpenAI(api_key="sk-fake-key-123")
try:
    bad_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": "hi"}],
    )
except openai.AuthenticationError:
    print("Login failed – check your API key.")

# part 2: cost comparison
short = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "Be helpful."},
        {"role": "user", "content": "What's the capital of France?"},
    ],
)

long_prompt = "You are a friendly, detailed assistant. " * 20
long = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": long_prompt},
```

```
        {"role": "user", "content": "What's the capital of France?"},  
    ],  
)  
  
print("Short prompt tokens:", short.usage.total_tokens)  
print("Long prompt tokens:", long.usage.total_tokens)
```

**Expected output:**

```
Login failed – check your API key.  
Short prompt tokens: 24  
Long prompt tokens: 168
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

### Approach 1 — the same comparison, in named functions

```
# auth_and_cost_practice.py
import openai
from openai import OpenAI

client = OpenAI()

def try_bad_key() -> None:
    bad_client = OpenAI(api_key="sk-fake-key-123")
    try:
        bad_client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": "hi"}],
        )
    except openai.AuthenticationError:
        print("Login failed – check your API key.")

def compare_prompt_cost(question: str) -> None:
    short_prompt = "Be helpful."
    long_prompt = "You are a friendly, detailed, thorough assistant who explains

    short = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": short_prompt}, {"role": "user", "c
    )
    long = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": long_prompt}, {"role": "user", "c
    )

    print(f"Short system prompt: {short.usage.total_tokens} tokens")
    print(f"Long system prompt: {long.usage.total_tokens} tokens")
```

```
try_bad_key()  
compare_prompt_cost("What's the capital of France?")
```

**Expected output:**

```
Login failed – check your API key.  
Short system prompt: 24 tokens  
Long system prompt: 187 tokens
```

**Difference from Basic:** splitting into `try_bad_key()` and `compare_prompt_cost()` makes each concern independently reusable and testable, and full type hints document what `compare_prompt_cost` needs (a question string) and that it doesn't return anything — it just prints.

**Which one should you actually write?** For this exercise, Intermediate's token-count comparison is enough — it directly proves Doc03's cost formula with real numbers. Turning `usage.prompt_tokens / completion_tokens` into an actual dollar figure (they're priced differently, so a bare token count can understate cost) is worth doing once you're reporting cost to someone else, not needed for this exercise or the Build Task.